AUTOSAR HLD Document Analysis Assistant: Technical Report

Name: ANBU SELVAN M

University: SASTRA Deemed University, Thanjavur

Register Number : 127003017

Summary

This project builds a locally run assistant that reads AUTOSAR High-Level Design (HLD) documents, extracts their architecture into structured data, and answers questions with page citations. It also checks the design for inconsistencies and compares document versions. Everything runs on one laptop CPU, with no calls to cloud services.

It was built and tested on a synthetic cruise-control HLD in two versions, with six planted design errors. On this data, extraction scored precision and recall of 1.0, the consistency checks found all six planted errors with no false positives, and the version comparison found all four changes. Cited question answering was fully correct on 10 of 14 new questions with the final design. The first, LLM-only design was fully correct on 5 of 10 original questions, and the final design on 9 of 10 of the same questions, which it was tuned on.

All results come from one synthetic document, so they show that the pipeline works but will probably overstate performance on real HLDs.

1. Problem statement and intended users

AUTOSAR High-Level Design documents are often hundreds of pages of unstructured PDF, so reviewers find mismatches, missing dependencies and inconsistent terminology by reading and cross-checking by hand. This is slow, depends on experienced specialists, and leaves no structured record of components, interfaces and signals. The project follows Case Study 1 of the Automotive Engineering AI case study document.

The objectives, taken from that case study, are:

- reduce the manual effort of reviewing an HLD
- build a structured, searchable record of architecture knowledge

- improve traceability between components, interfaces, signals and dependencies
- support design reviews, onboarding and impact analysis

Intended users: system and AUTOSAR architects, software developers, integration engineers, test engineers and engineering managers. The prototype has three access roles: viewer (ask and read), engineer (also review and correct) and admin (also read the audit log).

In scope (built):

- PDF ingestion, with an OCR fallback for scanned pages (implemented but untested, because the test document has no scanned pages)
- extraction of components, ports, interfaces and data elements
- natural-language questions with page and section citations
- rule-based inconsistency reports and a comparison of two document versions
- structured export, and a dependency graph for traceability

Out of scope: automatic approval of architecture decisions, changing source HLD documents without review, and replacing formal architecture governance.

2. Input data and knowledge base

The knowledge base is one synthetic cruise-control HLD in two versions, because real project HLDs are confidential and were not available. A script (`generate_synthetic_hld.py`) writes both versions as short PDFs with headings, a component catalogue, a ports table per component and a revision history. The AUTOSAR concepts used are software components, provided and required ports, port interfaces (Sender-Receiver and Client-Server), runnables with periods, and data elements with data types and units.

Version 1.0 has 5 components and 9 ports. Version 1.1 adds `SWC_BrakeManager` (6 components, 10 ports) and changes two periods.

Component	Kind	Runnable	Period v1.0 (ms)	Period v1.1 (ms)
SWC_WheelSpeedSensor	Sensor-actuator SWC	Rn_ReadWheelSpeed	10	5

Component	Kind	Runnable	Period v1.0 (ms)	Period v1.1 (ms)
SWC_VehicleSpeedCalc	Application SWC	Rn_CalcVehicleSpeed	10	10
SWC_CruiseControl	Application SWC	Rn_CruiseControlStep	20	10
SWC_EngineControl	Application SWC	Rn_ApplyThrottle	10	10
SWC_DisplayCluster	Application SWC	Rn_UpdateDisplay	100	100
SWC_BrakeManager	Sensor-actuator SWC	Rn_ReadBrake	not present	10

Six design errors are planted in version 1.0 and serve as the ground truth for the consistency checks. Version 1.1 fixes only the first.

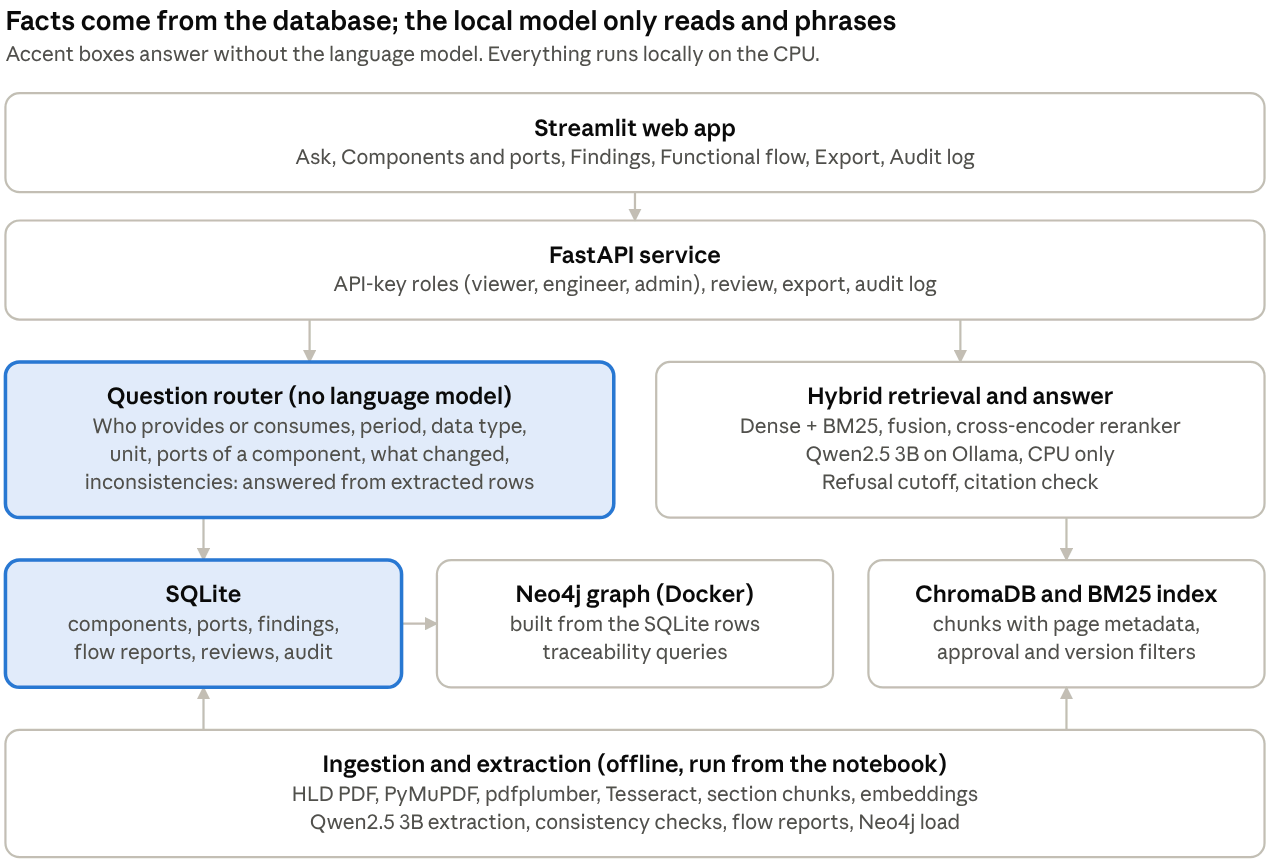
ID	Error type	Where
ISS-01	Missing provider	`SWC_CruiseControl.Rp_BrakeStatus` has no providing port
ISS-02	Interface type mismatch	`If_ThrottleRequest` is Sender-Receiver in one component and Client-Server in the other
ISS-03	Data type mismatch	`vehicleSpeed` is uint16 in two components and uint8 in `SWC_DisplayCluster`
ISS-04	Unit mismatch	`vehicleSpeed` is km/h in two components and mph in `SWC_DisplayCluster`
ISS-05	Unused provider	`SWC_VehicleSpeedCalc.Pp_VehicleAccel` is not used by any component
ISS-06	Inconsistent name	the text calls the component `SWC_VehSpeedCalc` instead of `SWC_VehicleSpeedCalc`

The ground truth (labelled components, ports, the six errors and the four version changes) is stored in `Input_Data/ground_truth`. Two question sets were drafted with an AI assistant: set 1 has 12 questions (10 answerable) and set 2 has 16 (14 answerable). Each includes questions the system should refuse.

No public AUTOSAR specification was ingested. Those documents describe the standard, not a project, so they have no component network and no ground truth. This is a limitation of the evaluation.

3. Solution architecture

The system splits the work by what each part does well: a small local language model reads documents and phrases open answers, while a database and rules supply facts that have an exact answer, such as who provides a signal. All parts follow the pilot stack named in the case study and run locally on a CPU.



system architecture · 5 layers, 3 stores

Structured questions go to the router and SQLite; open questions use retrieval and the local model; the stores are filled offline from the PDFs.

Layer	Choice	Role
Language model	Qwen2.5 3B (instruction-tuned) served by Ollama, CPU only	Extracts entities and writes answers to open questions
Embeddings	BAAI/bge-small-en-v1.5 (384 dimensions)	Semantic search over document chunks
Reranker	cross-encoder/ms-marco-MiniLM-L-6-v2	Reorders candidate chunks and drives the refusal cutoff
Vector store	ChromaDB	Chunk text, vectors and metadata filters (approval status, version)
Keyword search	rank-bm25	Exact identifiers such as `Rp_BrakeStatus`
Structured store	SQLite	Components, ports, findings, flow reports, review status, audit log
Graph store	Neo4j 5 in Docker	Port connections and traceability queries
Service layer	FastAPI	Role-based API: ask, review, export, audit
Interface	Streamlit	Questions, review tables, findings, flow reports, export, audit log
Orchestration	Custom Python	Routing, fusion, prompts, citation and grounding checks

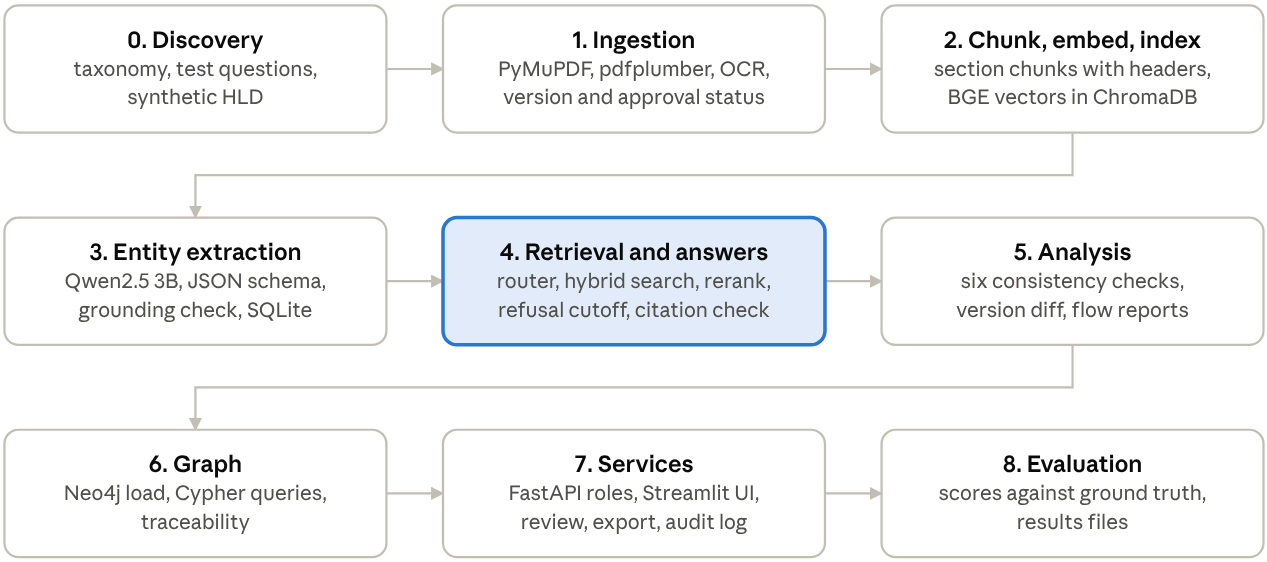
The pipeline is written directly in Python and does not use LangChain or LlamaIndex, so every step (chunking, fusion, refusal, citation check) is explicit code. FAISS, the alternative vector index in the case study, was considered and dropped because it has no metadata filtering, which the approved-version filter needs. A 7B model was tried only for speed and gave 2.7 tokens per second on this CPU, so the 3B model was used for all results.

4. Implementation and configuration

The system is built in nine phases. All code is in one Jupyter notebook (`Code/Code.ipynb`), which also writes the supporting files `qa_router.py`, `hld_core.py`, `api.py`, `app.py` and `generate_synthetic_hld.py`.

Nine phases turn a PDF into reviewed, traceable architecture data

The accent marks where each question is routed to the database or to the language model.



workflow · 9 phases

The phases run in order; phases 5 to 7 reuse the rows stored in phase 3, and phase 8 scores every stage.

Phase	What happens	Main tools
0. Discovery	Define the AUTOSAR entities to extract, the test questions and the synthetic HLD	Python, reportlab
1. Ingestion	Register each document with version, approval status and SHA-256 hash. Extract text, tables (kept as markdown), headings from font size, and page numbers; drop repeated headers; OCR fallback for scanned pages	PyMuPDF, pdfplumber, Tesseract

Phase	What happens	Main tools
2. Chunk, embed, index	Section-aware chunks of up to 900 characters, each starting with a header naming document, version, section path and pages. Embed and store in one ChromaDB collection per project	BGE embeddings, ChromaDB
3. Extraction	One component section per model call, output forced into a Pydantic JSON schema, retries on invalid output. Each value is checked against the source text, and rows are stored with review status pending	Qwen2.5 3B, Pydantic, SQLite
4. Retrieval and answers	Structured questions go to the database. Other questions use hybrid retrieval, a reranker, a refusal cutoff, a model answer from numbered evidence, and a citation check	SQLite, BM25, ChromaDB, cross-encoder
5. Analysis	Six consistency checks, a version comparison and flow reports built from the extracted rows	Python, SQLite
6. Graph	Load documents, components, ports, interfaces and findings into Neo4j and run traceability queries	Neo4j, Cypher
7. Services	API with three roles, review of every extracted row, export and audit log, plus the web interface	FastAPI, Streamlit
8. Evaluation	Score each stage against the ground truth and write results files	Python, pandas

The six consistency checks compare ports that use the same interface name: a required port with no provider, a provided port nobody uses, different interface types, different data types, different units, and a component name in the text that is not a defined component. Treating a shared interface name as a connection is an assumption, because the HLD text does not list the wiring explicitly.

Configuration

Setting	Value
Hardware	Laptop with 16 GB RAM and 12 CPU cores; no GPU used
CPU-only controls	GPU hidden from the Ollama server and the API, Ollama `num_gpu` set to 0, embedding and reranker models on the CPU
Model settings	Temperature 0, context 4096 tokens, schema-constrained JSON output
Retrieval	Top 10 from dense and BM25 search, reciprocal rank fusion (k = 60), rerank, keep top 3
Refusal cutoff	Best rerank score below -5.0 means out of scope; no model call is made
Extraction time	About 30 to 65 seconds per component, about 4 minutes per document
Docker	Neo4j, plus the API and web app as two containers (docker compose in Code), with host networking to reach Ollama

5. Evaluation steps and results

Every result below comes from a file in `Evaluation_Results` and was measured on the synthetic HLD. Each stage was scored against the ground truth defined in `Input_Data/ground_truth`.

Stage	Method	Result
Entity extraction	All fields of each component and port must match the labelled data exactly	v1.0: 5 of 5 components, 9 of 9 ports. v1.1: 6 of 6 and 10 of 10. Precision and recall 1.0
Retrieval	Is the section that holds the evidence in the top 3 for each of 10 answerable questions?	Dense search only: 0.9. Hybrid: 1.0. Hybrid with reranker: 1.0

Stage	Method	Result
Consistency checks	Compare findings with the six planted errors	v1.0: 6 of 6 found, no false positives. v1.1: the brake provider error gone, the other five remain
Version comparison	Compare the reported changes with the four known changes	4 of 4 found, no extra changes
Graph	Compare missing and unused provider counts from Neo4j with the SQLite checks	Equal in both versions
Flow summaries written by the model	Compare each summary with the extracted facts	6 of 9 accurate

Question answering was run three times. Run 1 used the language model for every question. Run 2 sends structured questions to the database and adds the citation check. An answer counts as correct only if it matches the document, and a citation counts as correct only if the cited section holds the evidence.

Run	Answer correct	Citation correct	Both correct	Refusals correct
Run 1, set 1 (model only)	7 of 10	7 of 10	5 of 10	2 of 2
Run 2, set 1 (fixes designed after seeing these failures)	10 of 10	9 of 10	9 of 10	2 of 2
Run 2, set 2 (new questions, written after the router was designed)	12 of 14	11 of 14	10 of 14	1 of 2

Grades were proposed by the AI assistant and checked by the student against the HLD. The set 1 row of run 2 is not a fair test, because the fixes were built after seeing those failures. Set 2 is the better figure, though not an independent one.

Faithfulness sample. A RAGAS-style faithfulness score was computed with an own implementation of the method: the model splits each answer into statements and judges each one against the retrieved evidence. The same 3B model did both jobs, so the scores

are rough. Only answers written by the model were judged, which is why run 2 has few answers: the router answers the other questions from the database.

Run	Model answers judged	Statements	Supported	Faithfulness
Run 1, set 1	10	28	16	0.55
Run 2, set 1	1	3	2	0.67
Run 2, set 2	4	13	7	0.52

Reading the verdicts by hand showed two sources of error in this score. The splitter sometimes added statements the answer never made, and the judge sometimes marked a statement as unsupported although the evidence states it. The scores therefore understate faithfulness, and the run 2 rows rest on too few answers to compare with run 1.

What went wrong

- In run 1 the model named a component as the provider of the brake status, when no component provides it. A later test in the web app gave a different wrong component.
- The set 2 failures: the router answered only half of a comparison question; a question phrased as "which components changed their timing" was refused although it was answerable; and a question about a CAN message, which the document does not mention, got a wrong answer that the citation check marked unverified, so it counts as a missed refusal.
- The citation check passed all nine model-written flow summaries, including the three that were wrong, because it checks that names and numbers exist in the source, not that they are connected correctly.

Limits of this evaluation

- One synthetic document built from a template, so the perfect extraction and check scores mainly show that the pipeline is wired correctly.
- The router and the checks were written knowing the planted errors.
- Small samples: 10 to 14 questions and 21 chunks.
- The faithfulness score comes from a small model judging its own outputs, and the time saved against manual review, a metric the case study suggests, was not measured.

6. Responsible AI measures

The design aims for two things: no answer reaches an engineer without its source, and no AI output becomes a decision without human review.

Grounding

- Open questions are answered only from three numbered evidence chunks, and the prompt tells the model to answer "not found" when the evidence does not contain the answer.
- Structured questions are answered from extracted database rows and labelled as a database lookup.
- A cutoff on the reranker score refuses out-of-domain questions without calling the model. In testing, the best chunk of three answerable questions scored 0.6 to 7.0 and the France question scored -11.3. One answerable question was refused wrongly (section 5).
- Every extracted value is checked against the source text, and none of the extracted rows were ungrounded.
- A citation check compares the names and numbers in a model answer with the cited text. A failure sets the status to unverified and the app shows a warning.
- Limits: the confidence label reflects retrieval strength only, and the citation check cannot detect swapped relationships.

Privacy

- All models run locally on the CPU. Document text, embeddings, prompts and answers are not sent to any external service.
- The test data is synthetic, so no confidential document is involved.
- The Neo4j password is typed at run time and not written to a file. Demo API keys are stored locally in `Code/store/users.json`.

Security

- API keys map to three roles: viewer, engineer and admin. Only engineers can review, and only admins can read the audit log.
- Questions are limited to 3 to 500 characters, reviews can edit only whitelisted fields, and direction and interface type values are validated.
- Database queries use parameters and fixed table names.
- Document text is passed to the model as evidence, and the prompts tell it to ignore instructions inside that text. This was not tested with adversarial documents.

- Only documents marked approved are searched, and the API listens on localhost only.
- Limit: demo keys are not real identity management, and the prototype holds one project.

Human oversight

- Every extracted component, port and finding has a review status (pending, accepted, edited or rejected) that engineers set in the web app. Edits record the old and new values.
- Rejected rows are left out of answers and exports.
- The app states that output must be reviewed by a qualified engineer before use in design, compliance or release. No step approves an architecture decision automatically.

Audit trail

- An audit log records document registration (with hash), ingestion, indexing, extraction, questions, reviews, exports and graph builds.

7. Innovation highlights

The main idea is to let the language model do only what it is good at, and to compute everything else.

- **Question routing.** Structured question types (who provides, who consumes, period, data type, unit, what changed, what is inconsistent) are answered from extracted rows. This removed the confident wrong answers on "who provides" questions.
- **Grounded extraction.** Output is forced into a JSON schema, retried when invalid, and every value is checked against the source text.
- **Citation check.** An answer whose names or numbers are missing from its cited text is marked unverified.
- **Identifier-aware keyword search.** The BM25 tokenizer keeps identifiers such as `Rp_BrakeStatus` whole and also splits them into parts.
- **Section-aware chunks.** Each chunk starts with a header naming document, version, section path and pages, and tables stay whole as markdown.
- **AUTOSAR checks and version comparison.** Six rules over provided and required ports, plus a comparison that lists components added, values changed, and findings resolved or introduced.

- **Grounded flow reports.** The model-written summaries were accurate in 6 of 9 cases, so the delivered reports join the document's own purpose sentence to provider and consumer links computed from the ports.
- **Graph traceability.** Cypher queries find missing providers, everything downstream of a component, and everything upstream of it, and agree with the SQL checks.
- **Prompt design.** Prompts ask for schema-constrained JSON, allow only the supplied evidence, require a "not found" answer, and treat document text as data, not instructions.
- **CPU-only operation.** The whole pipeline runs on a 16 GB laptop with local models and no GPU.

8. Limitations and future work

The results should be read with these limits in mind.

- **One synthetic document.** No real HLD and no public AUTOSAR specification was tested, so real-world accuracy is unknown and probably lower.
- **Keyword router.** Questions that are phrased unusually fall through to the language model, and one comparison question received only half an answer.
- **Refusal cutoff.** A single score threshold produced one false refusal and one missed refusal in set 2.
- **Connection rule.** Ports are treated as connected when they share an interface name, whereas real HLDs and AUTOSAR models define connections explicitly.
- **Small model.** The 3B model is weak on absence, direction and multi-step questions, as the flow summaries showed.
- **Untested parts.** The OCR fallback was not exercised, and diagrams are not interpreted: only text and tables are extracted, so the boxes and arrows of a diagram are lost, and so is text inside an image on a page that has other text. Review-time saving was not measured.
- **Prototype scope.** New documents are ingested from the notebook, not the web app; API keys are demo keys, not identity management; there is one project; and the containers mount the project folder and reach Ollama on the host, so the package is not a standalone deployment.

Future work, in order of value: evaluate on a real HLD obtained through an industry partner, parse explicit port connections, test a larger model on the failing question types, add document upload and project isolation with real identity management, and make the

containers self-contained by including the model server.

9. External sources and tool declaration

- **Project brief:** *Automotive Engineering AI: Standard Project Case Study Document*, version 1.0, September 2026 (Case Study 1).
- **Terminology:** AUTOSAR concepts such as software components, ports and port interfaces. No AUTOSAR document was ingested.
- **Models:** Qwen2.5 3B (Qwen team, Alibaba Cloud) served with Ollama; BAAI/bge-small-en-v1.5; cross-encoder/ms-marco-MiniLM-L-6-v2.
- **Software:** PyMuPDF, pdfplumber, Tesseract with pytesseract, sentence-transformers, ChromaDB, rank-bm25, Pydantic, FastAPI, Uvicorn, Streamlit, SQLite, Neo4j with its Python driver, Docker, pandas, reportlab and Jupyter.
- **AI assistance:** Claude (Anthropic) was used to draft the notebook code, the synthetic HLD generator, the question sets, proposed grades and this report. The signed declaration in the `Declarations` folder lists each tool, the artifact it affected and how the student verified it.